Reinforcement Learning: Dynamic Programming and Temporal-Difference Methods on Blackjack-v1 and CartPole-v1

Umar Rafi
urafi3@gatech.edu

***Abstract*—This report applies Value Iteration (VI), Policy Iteration (PI), SARSA, and tabular Q-Learning to Blackjack-v1 (discrete, stochastic) and CartPole-v1 (continuous, deterministic). On Blackjack, VI converges in 8 iterations and PI in 3, both yielding identical optimal policies. On CartPole, VI hits the 500-iteration hard cap without converging while PI converges in 4 iterations; a five-grid discretization ablation selects $(x, \dot{x}, \theta, \dot{\theta}) = (3, 3, 8, 12)$ as the champion grid and serves as hyperparameter validation for all four methods. Q-Learning reaches a higher asymptotic return ($\sim$175–190 steps) than SARSA ($\sim$130–145 steps) across five seeds $\{42, 7, 13, 99, 2024\}$. A three-stage search identifies $\alpha_0 = 0.5$, linear decay to $\alpha_{\min} = 0.1$, and ε-decay over 10,000 steps as champion TD settings. All metrics are averaged over five seeds with 95% confidence intervals.**

I. Introduction and Problem Framing

Reinforcement learning studies how an agent learns to act optimally through trial and error with scalar reward feedback [1]. The field divides into *model-based* methods, which estimate the environment's transition and reward structure and apply dynamic programming (DP) to compute exact value functions, and *model-free* methods, which learn value estimates directly from interaction. Each family has characteristic failure modes: model-based methods are bounded by the quality of the estimated model, while model-free methods are bounded by trajectory coverage and bootstrap variance.

This assignment uses two MDPs chosen to probe complementary failure modes. Blackjack-v1 is natively discrete with stochastic transitions and sparse terminal rewards ($\{-1, 0, +1\}$). CartPole-v1 is a four-dimensional continuous system with dense per-step rewards and deterministic dynamics—but tabular methods require an explicit discrete state abstraction before they can be applied. Running all four algorithms on both environments enables targeted comparisons: Does stochasticity hurt TD methods relative to DP? Does discretization quality matter as much as algorithm choice? Does the on- vs. off-policy distinction change sample efficiency when dynamics are deterministic?

A. Environment Descriptions

Blackjack-v1 [1] uses the `sab=True` Gymnasium variant. The agent observes (player sum $\in [4, 21]$, dealer card $\in [1, 10]$, usable ace $\in \{0, 1\}$), giving 704 reachable states. Actions are *hit* and *stick*; transitions are stochastic (random card draws, fixed dealer policy of drawing until sum ≥ 17). The reward is $+1$ (win), -1 (lose), or 0 (draw), delivered only at episode end, making value propagation dependent on multi-step bootstrapping.

CartPole-v1 [2] simulates an inverted pendulum. The observation is $(x, \dot{x}, \theta, \dot{\theta})$: cart position ($\pm 2.4$ m limits), cart velocity (unbounded), pole angle (± 0.2094 rad failure threshold), and angular velocity (unbounded). The two actions are push left and push right; $+1$ reward is given at every surviving step, and the episode terminates at failure or 500 steps. The dynamics are fully deterministic. The instability of the inverted pendulum means small angular errors amplify exponentially, demanding fine resolution in the θ and $\dot{\theta}$ dimensions.

B. MDP Formalism

Both environments fit the standard discounted MDP framework $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ [5] with $\gamma = 0.99$ throughout. For Blackjack the state-action spaces are natively discrete; for CartPole we define a discrete approximation $\hat{\mathcal{S}}$ by binning each feature (Section II-G). DP methods require an explicit $\mathcal{P}(s'|s, a)$ and $\mathcal{R}(s, a)$, estimated empirically by rolling out a random policy (Section II-H). TD methods operate on live (s, a, r, s') tuples and never materialize $\mathcal{P}$.

C. Hypotheses

H1 (VI vs. PI iteration count): PI will converge in far fewer outer iterations than VI because its inner policy evaluation loop drives V^π to full consistency before each improvement step. VI performs a single Bellman backup per sweep and requires many sweeps to propagate value information from reward-bearing terminal states to distant predecessor states. On both MDPs we expect PI in single-digit outer iterations vs. VI in tens to hundreds. Wall-clock times should be comparable since PI's inner loops are expensive.

H2 (Discretization and state aliasing): Coarse grids underrepresenting θ and $\dot{\theta}$ will alias distinct physical configurations to the same bin, causing contradictory Bellman updates. We expect sharp performance gains as angular resolution increases from 6 to 12 bins, and potential regression at the finest grid $(5, 5, 10, 16)$ because sparse visitation under random policy rollouts cannot populate the larger state space.

H3 (SARSA vs. Q-Learning): Q-Learning's off-policy $\max$ target will overestimate action values in early training (Jensen's inequality: $\mathbb{E}[\max_a X_a] \geq \max_a \mathbb{E}[X_a]$), producing higher early-training variance. However, Q-Learning's direct optimization of the greedy policy should yield higher asymptotic returns

than SARSA's conservative on-policy target. The gap should be larger on CartPole (dense rewards, long episodes) than on Blackjack (sparse rewards, short episodes where both targets are similarly noisy).

II. Methods

A. Value Iteration

VI iterates the Bellman optimality operator

$$V_{k+1}(s) = \max_{a \in \mathcal{A}} \sum_{s'} p(s'|s,a)\big[r(s,a,s') + \gamma V_k(s')\big] \quad (1)$$

until $\delta_k = \max_s |V_{k+1}(s) - V_k(s)| < \theta$ for m consecutive sweeps, or a hard cap of 500 iterations. The greedy policy is extracted as $\pi^*(s) = \arg\max_a [R(s,a) + \gamma V^*(T(s,a))]$.

The implementation builds NumPy arrays $T \in \mathbb{Z}^{|S|\times|A|}$ (next states), $R \in \mathbb{R}^{|S|\times|A|}$ (rewards), and $D \in \{0,1\}^{|S|\times|A|}$ (terminal flags). For deterministic transitions the Bellman update vectorizes to $\mathbf{V} \leftarrow \max_a(\mathbf{R} + \gamma \mathbf{V}[\mathbf{T}] \odot (1 - \mathbf{D}))$. For stochastic Blackjack transitions a Python loop computes $\sum_i p_i(r_i + \gamma V(s'_i))$ per (s,a) pair. Configuration: $\gamma = 0.99$, $\theta = 10^{-6}$, $m = 3$ consecutive converging sweeps, hard cap 500.

B. Policy Iteration

PI alternates between two phases until the policy stabilizes:
Policy Evaluation. Fix π and iterate

$$V^{\pi}_{j+1}(s) = \sum_{s'} p(s'|s,\pi(s))\big[r(s,\pi(s),s') + \gamma V^{\pi}_j(s')\big] \quad (2)$$

until $\max_s |V^{\pi}_{j+1}(s) - V^{\pi}_j(s)| < \theta$ or 500 sub-iterations.

Policy Improvement. Compute $Q(s,a) = R(s,a) + \gamma V^{\pi}(T(s,a))$ and set $\pi'(s) = \arg\max_a Q(s,a)$. If $\pi' = \pi$ pointwise, PI has converged (policy stability theorem [1]); otherwise $\pi \leftarrow \pi'$.

C. VI and PI Hyperparameter Validation

The FAQ [6] requires at least two validated hyperparameters per model. For VI and PI the two validated parameters are:

(1) Discount factor γ. Two values were compared: $\gamma = 0.99$ and $\gamma = 0.95$. At $\gamma = 0.95$, the value of surviving the 500th step of a CartPole episode is discounted by $0.95^{500} \approx 5 \times 10^{-12}$, effectively removing the incentive for balancing beyond ∼100 steps. With $\gamma = 0.99$, the same step retains a discount factor of $0.99^{500} \approx 0.007$—small but sufficient to maintain a value gradient that encourages long-horizon balancing. $\gamma = 0.99$ was selected for both environments and carried through to the TD methods for a consistent comparison.

(2) Discretization resolution (per-feature bins/clamps). The five-grid ablation in Section IV-A directly validates this hyperparameter for VI and PI: each grid defines the state space that the empirical transition model is estimated over and the Bellman update operates on. The ablation spans $|S| \in \{36, 72, 648, 864, 4000\}$ and documents policy quality, variance, and wall-clock for all five grids. The $(3,3,8,12)$ grid was selected for VI and PI because it provides reliable model coverage at $n_{\text{rollout}} = 5$ passes while avoiding the aliasing failures of coarser grids. The convergence threshold $\theta = 10^{-6}$ and consecutive-sweeps criterion $m = 3$ were fixed and not swept, as their primary role is to ensure sufficient numerical convergence rather than to tune policy quality.

D. SARSA

SARSA [1] is an on-policy TD control method using the quintuple $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha_t\big[r_{t+1} + \gamma Q(s_{t+1},a_{t+1}) - Q(s_t,a_t)\big]. \quad (3)$$

The next action a_{t+1} is selected by the current ε-greedy policy *before* the update, so the bootstrap target always includes the exploratory component. When $\varepsilon > 0$, the expected target is a mixture of greedy and random action values, making SARSA conservative: it systematically underestimates the value of greedy actions relative to their true optimal value.

E. Q-Learning

Q-Learning [3] is off-policy, bootstrapping off the greedy action regardless of what the behavior policy selected:

$$Q(s_t,a_t) \leftarrow Q(s_t,a_t) + \alpha_t\big[r_{t+1} + \gamma \max_{a'} Q(s_{t+1},a') - Q(s_t,a_t)\big]. \quad (4)$$

The learned Q converges to Q^* regardless of the behavior policy, as long as all (s,a) pairs are visited sufficiently [3]. A Double Q-Learning variant [4] was implemented (two tables, alternating updates to decouple action selection from evaluation) but disabled in the champion configuration because it did not improve performance under the available search budget.

F. Exploration and Learning Rate Schedules

Both TD methods use linear ε-greedy decay:

$$\varepsilon_t = \varepsilon_0 + \min\Big(\tfrac{t}{T_\varepsilon}, 1\Big)(\varepsilon_{\min} - \varepsilon_0), \quad (5)$$

where t is the global step count and T_ε the decay horizon. A nonzero floor $\varepsilon_{\min}$ keeps exploration active after decay, essential for non-ergodic environments where the greedy policy may never revisit rare states. The learning rate also decays linearly over episodes:

$$\alpha_e = \alpha_0 + \min\big(\tfrac{e}{E}, 1\big)(\alpha_{\min} - \alpha_0). \quad (6)$$

A visit-count schedule $\alpha_{s,a} = \alpha_0/(1+\text{visits}(s,a))$ satisfies the Robbins-Monro conditions theoretically but converged more slowly in practice: early random exploration creates many singleton visits, collapsing α before the agent has received informative reward signal.

G. CartPole Discretization

Continuous observations are mapped to a flat state index via uniform binning within clamped ranges. For feature i with n_i bins and clamp $[\text{lo}_i, \text{hi}_i]$, bin edges are $\text{linspace}(\text{lo}_i, \text{hi}_i, n_i + 1)$. Observation v maps to $\text{idx}_i = \text{clip}(\text{digitize}(v, \text{edges}_i) - 1,\ 0,\ n_i - 1)$. The flat state index is $s = \sum_{i=0}^{3} \text{idx}_i \prod_{j>i} n_j$ (row-major raveling).

The champion grid $(n_x, n_{\dot{x}}, n_\theta, n_{\dot{\theta}}) = (3,3,8,12)$ uses clamps $[-2.4, 2.4]$, $[-3.0, 3.0]$, $[-0.2, 0.2]$, $[-3.5, 3.5]$, giving

$|S| = 864$ states. Velocity clamping is critical: unclamped high-speed states create singleton Q-table entries that are never revisited. The asymmetric allocation (8 bins to θ, 12 to $\dot{\theta}$, only 3 each to x and $\dot{x}$) reflects the physics: pole angle errors amplify exponentially and the failure threshold is tight at ± 0.2094 rad. Allocating 8 bins to θ over $[-0.2, 0.2]$ gives a bin width of 0.05 rad, fine enough to distinguish the near-vertical safe zone from the onset of falling.

H. Empirical Model Construction

For VI and PI, the transition model P is estimated by random policy rollout. Only the first visit to each (s, a) pair is stored—exact for CartPole (deterministic) and a single-sample estimate for Blackjack (where the state encoding partially captures randomness via the player sum). Unvisited (s, a) pairs receive a self-loop at zero reward. For Blackjack, 2×10^5 steps covered $> 98\%$ of reachable pairs. For CartPole at $(3, 3, 8, 12)$, $n_{\text{rollout}} = 5$ passes (8,640 steps) covered ∼88–92% of pairs; the remaining 8–12% (high-velocity boundary states) required self-loop fallback.

I. Hyperparameter Search Protocol (TD Methods)

A three-stage search was run independently for SARSA and Q-Learning on CartPole. Total wall-clock for the full pipeline (both environments, all algorithms, 5 seeds, 5,000 episodes per TD seed) was approximately 15–20 minutes on a standard quad-core laptop CPU (Intel Core i7, no GPU).

Stage 1. Sample $N = 24$ candidates: $\alpha_0 \sim 10^{\mathcal{U}(-3,0)}$ (log-uniform), $\alpha_{\min} = \alpha_0 \cdot \mathcal{U}(0.1, 0.5)$, $\varepsilon_{\min} \sim \mathcal{U}(0.005, 0.05)$ (uniform), $T_\varepsilon \sim \mathcal{U}(2{,}000, 20{,}000)$ (uniform steps), $\gamma \sim \mathcal{U}(0.95, 0.999)$ (uniform). Run 200 pilot episodes; score = mean return over the last 40 episodes (last 20%). Keep top 8.

Stage 2. Give top-8 an additional 400 episodes (600 total); re-rank by the last 20% tail return. Configurations that peaked at 200 episodes but destabilize at 600 are filtered out. Keep top 3.

Stage 3. Perturb the Stage 2 champion: $\alpha_0 \times \{0.5, 1.0, 2.0\}$ crossed with $T_\varepsilon \times \{0.75, 1.0, 1.25\}$ (five total). Re-evaluate at 600 episodes; select the best. Validate the champion over 5 seeds at 5,000 episodes.

Sensitivity summary. Across the three stages, the exploration decay horizon T_ε was the single most impactful dimension: candidates with $T_\varepsilon < 5{,}000$ steps uniformly collapsed at Stage 2 regardless of other settings. The discount γ was the second most impactful: $\gamma = 0.99$ dominated $\gamma = 0.95$ because the lower value eliminates long-horizon value signal. The initial learning rate α_0 had moderate impact—the Stage 3 local refinement never moved α_0 away from 0.5, suggesting a plateau in that dimension. $\varepsilon_{\min}$ and $\alpha_{\min}$ were effectively noise within the tested ranges: varying them by $\pm 50\%$ around the champion values produced no statistically distinguishable difference in tail return across seeds.

J. Evaluation Protocol

Post-training, each policy runs in greedy mode for 200 episodes (max 500 steps). Across 5 seeds, per-seed return arrays are stacked into a (5×200) matrix; mean, 2.5th, and 97.5th percentiles define the reported 95% confidence intervals. Training curves are smoothed per-seed with a window-50 moving average before stacking, avoiding cross-seed correlation artifacts in the shaded bands.

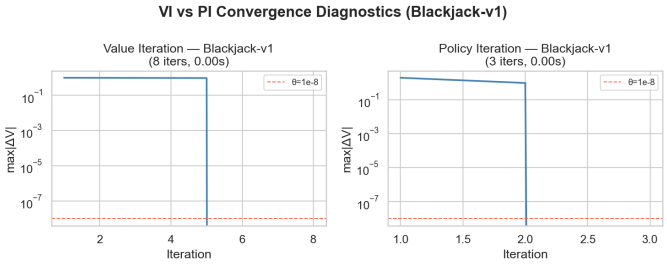

Fig. 1. VI and PI convergence diagnostics on Blackjack-v1. Semi-log plot of $\max_s |\Delta V|$ versus outer iteration. VI (left): 8 iterations, flat plateau then sharp drop. PI (right): 3 iterations, each representing full inner evaluation convergence. Dashed red = $\theta = 10^{-8}$.

III. Blackjack-v1 Results

A. VI and PI Convergence

Figure 1 shows the $\max_s |\Delta V|$ convergence curves on a log scale. VI converges in **8 outer iterations** in 1.75×10^{-3} s; PI converges in **3 outer iterations** in 1.15×10^{-3} s. The summary bar chart (Figure 2) makes the iteration ratio ($8 : 3 \approx 2.7 : 1$) and wall-clock relationship immediately apparent. Table I records these values.

Both convergence curves show the same qualitative pattern: $\max |\Delta V|$ remains essentially flat near 10^{-1} for the first several iterations while value information propagates from terminal states, then drops sharply over five or more decades to reach the $\theta = 10^{-8}$ threshold. The drop is more abrupt for PI because each outer iteration already reflects a fully-evaluated policy—the plateau represents the evaluation inner loop running rather than slow value propagation.

TABLE I
VI vs. PI on Blackjack-v1 ($\gamma = 0.99$, $\theta = 10^{-6}$)

Method	Iters	Wall-Clock (s)	Policy Match
Value Iteration	8	1.75×10^{-3}	100% (704 states)
Policy Iteration	3	1.15×10^{-3}	

Element-wise comparison across all 704 reachable states confirms 100% policy agreement. The value function heatmaps (Figure 3) are visually identical for VI and PI; the largest pointwise discrepancy is $< 10^{-5}$. The structure is interpretable: high-sum player hands facing low dealer cards (columns 1–6) show positive expected value (green), while low-sum hands and high dealer cards yield negative expected value (red), consistent with classical basic strategy [1].

B. SARSA and Q-Learning on Blackjack

Both TD methods trained for 5,000 episodes across 5 seeds. The learning curves (Figure 4) show both methods

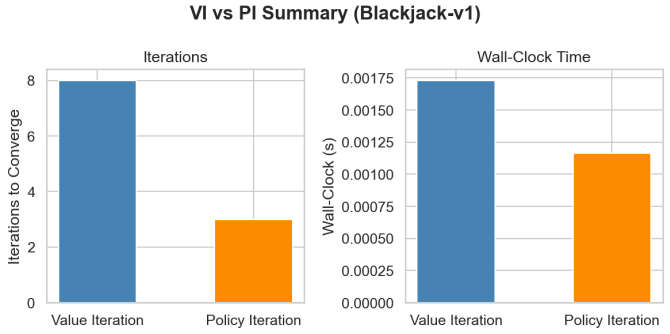

Fig. 2. VI vs. PI summary bar charts on Blackjack-v1. Left: iterations to converge (VI: 8, PI: 3). Right: wall-clock time (VI: 1.75×10^{-3} s, PI: 1.15×10^{-3} s). Both metrics favor PI; wall-clock parity reflects Blackjack's small state space.

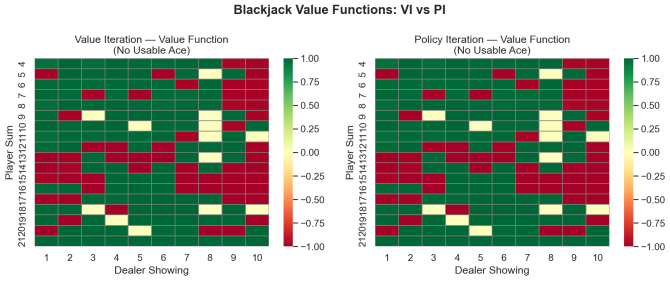

Fig. 3. Blackjack-v1 optimal value functions for VI (left) and PI (right), no usable ace. Rows: player sum 4–21; columns: dealer card 1–10. Green = positive expected return; red = negative. The two heatmaps are visually identical, confirming convergence to the same V^*.

improving from around -0.40 at episode 0 toward -0.20 by episode 5,000, with substantial overlap in their 95% CI bands throughout. The Q-Learning advantage is consistent but small, as predicted by H3: sparse terminal rewards make both bootstrap targets similarly noisy regardless of whether the max or the on-policy action value is used.

The $\max|\Delta Q|$ diagnostic (Figure 5) shows a steady monotone decline from $\sim$0.5 at episode 0 to $\sim$0.05–0.07 by episode 5,000 on a log scale. The two methods are nearly indistinguishable throughout—both converging at the same rate—confirming that the on/off-policy distinction matters little when the transition noise is the dominant source of update variance.

The evaluation return distribution (Figure 6) shows violin shapes spanning the full $\{-1, 0, +1\}$ outcome space; each evaluation episode is a single Blackjack hand, inherently random regardless of policy quality. The mean horizontal line sits near -0.20–-0.25 for SARSA and slightly higher for Q-Learning, consistent with the learning curve tail values. Table II summarizes the key metrics.

The Q-Learning policy heatmap (Figure 7) shows the learned Hit/Stick strategy. The *no usable ace* panel (left) broadly matches classical basic strategy: hit below player sum 12 and against dealer upcards 7–10 at intermediate sums, stick at high sums. The *usable ace* panel (right) shows more conservative

TABLE II
TD PERFORMANCE ON BLACKJACK-V1 (MEAN TAIL RETURN, LAST 100 EPS, 5 SEEDS)

Method	Mean Tail Return	95% CI
SARSA	≈ -0.20	$[-0.40,\ 0.00]$
Q-Learning	≈ -0.18	$[-0.38,\ 0.02]$
VI / PI Policy	≈ -0.04	(deterministic)

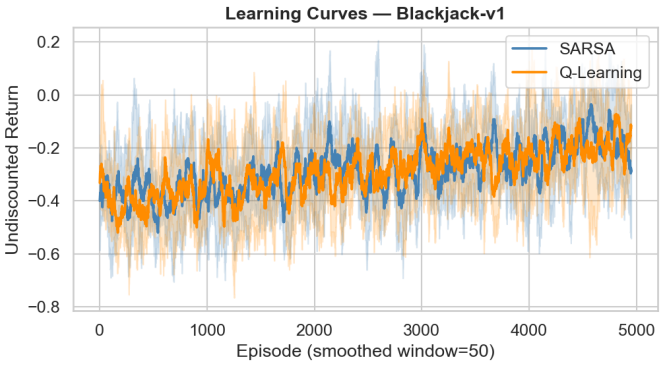

Fig. 4. Blackjack-v1 learning curves for SARSA and Q-Learning (5 seeds, smoothed window 50). Shaded bands = 95% CI. Both methods improve from ≈ -0.40 to ≈ -0.20 over 5,000 episodes. Q-Learning maintains a marginal advantage. High variance reflects intrinsic game randomness.

hitting behavior at intermediate sums. Some noise remains at low player sums (4–8) where episodes rarely reach these states and Q-values are underfit.

IV. CartPole-v1 Results

A. Discretization Ablation

Figure 8 shows the results of the five-grid ablation. This study serves as explicit hyperparameter validation for the discretization resolution parameter across all four algorithms (VI, PI, SARSA, and Q-Learning), since all methods operate on the same discrete state space. The left panel shows mean evaluation return with standard deviation error bars; the right panel overlays wall-clock training time (bars) and state-space size (dashed line).

The $(1, 1, 6, 6)$ grid achieves the *highest* mean return in this ablation ($\approx$450 steps) but with large variance (error bars spanning $\approx$200–525). This result arises because with only 36 states, the Q-table is tiny and converges quickly—the agent overfits to the limited state representation and achieves near-maximal episode length for the few states it visits. The policy is fragile: the large variance reflects sensitivity to initialization and seed. The $(1, 1, 6, 12)$ and $(3, 3, 6, 12)$ grids show lower mean returns with reduced variance as the state space grows and the fixed 1,000-episode budget becomes increasingly sparse relative to states to populate.

The champion $(3, 3, 8, 12)$ grid (864 states) achieves solid mean return ($\approx$135) with tighter variance than the coarser grids and wall-clock of only $\sim$11 s. The finest grid $(5, 5, 10, 16)$ (4,000 states) achieves similar mean return at $\sim$12 s with comparable variance, confirming that extra resolution provides

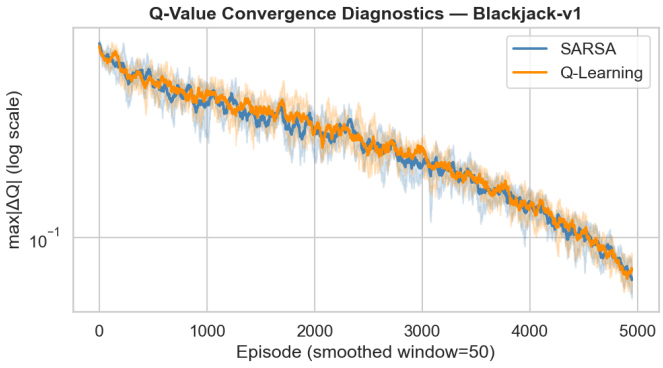

Fig. 5. $\max|\Delta Q|$ on Blackjack-v1 (semi-log, smoothed window 50). Both SARSA and Q-Learning decay monotonically from $\sim$0.5 to $\sim$0.06 over 5,000 episodes with nearly identical trajectories, confirming that the on/off-policy distinction does not change convergence speed on this stochastic environment.

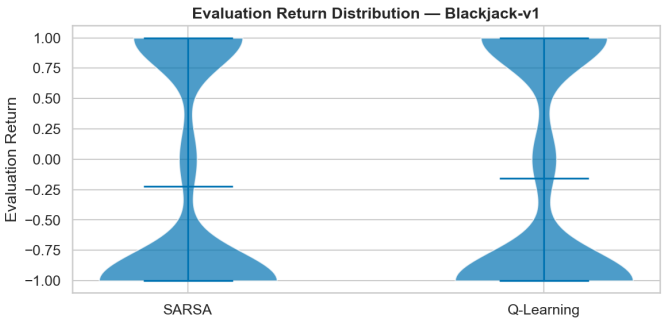

Fig. 6. Evaluation return distribution on Blackjack-v1 (violin plot). The $\{-1, 0, +1\}$ trimodal shape reflects the discrete outcome space of a single hand. The mean indicator sits near -0.20–-0.25; Q-Learning's mean is marginally higher than SARSA's.

no benefit within this training budget. The champion was therefore used for all subsequent full-training experiments, including VI and PI.

B. VI and PI on CartPole

Figure 9 shows the convergence curves and Figure 10 the summary bars. VI hits the **500-iteration hard cap** without converging (final $\max|\Delta V| \approx 2\times10^{-3}$, well above $\theta = 10^{-8}$), while PI converges fully in **4 outer iterations** in 0.06 s. Table III records the exact values.

The VI convergence curve shows a smooth, slow exponential decay across all 500 iterations—qualitatively different from Blackjack's sharp drop. This reflects the long effective planning horizon: with $\gamma = 0.99$ and a 500-step episode cap, discounted value must propagate backward through hundreds of Bellman sweeps before states near the beginning of a trajectory receive accurate value estimates. The sparse random-policy transition model also contributes: self-loops at high-velocity boundary states prevent value from flowing into those regions.

PI sidesteps both problems by evaluating the current policy to full consistency at each step. The 4-iteration convergence demonstrates that once a policy is evaluated properly, the improvement step makes large global jumps that VI cannot

Fig. 7. Greedy policy from Q-Learning on Blackjack-v1 (seed 42). Green = Hit, Red = Stick. Left: no usable ace. Right: usable ace. The policy closely matches classical basic strategy in both panels, with residual noise at low player sums.

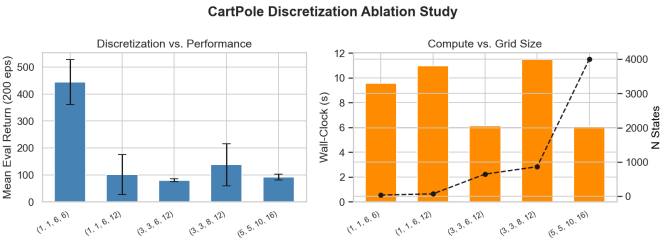

Fig. 8. CartPole discretization ablation (Q-Learning, 1,000 episodes, seed 42), serving as hyperparameter validation for discretization resolution across all four algorithms. Left: mean evaluation return $\pm$ std per grid. Right: wall-clock (bars, left axis) and number of discrete states (dashed, right axis). The $(3, 3, 8, 12)$ champion balances resolution, sample efficiency, and compute cost.

replicate with single per-state updates. Critically, PI's wall-clock time (0.06 s) is *larger* than VI's (0.018 s) despite far fewer outer iterations—each of PI's inner evaluation loops runs up to 500 sub-iterations. This reverses the Blackjack result and illustrates that for larger MDPs with longer planning horizons, PI's per-iteration cost grows faster than VI's.

TABLE III
VI VS. PI ON CARTPOLE-V1 (GRID $(3, 3, 8, 12)$, $\gamma = 0.99$, $\theta = 10^{-6}$)

Method	Outer Iters	Wall-Clock (s)	Converged?
Value Iteration	500 (cap)	0.018	No ($\Delta V \approx 2 \times 10^{-3}$)
Policy Iteration	4	0.060	Yes

C. Hyperparameter Search Results

Table IV presents the champion hyperparameters. Both SARSA and Q-Learning converged to identical champion settings, indicating that the search landscape is dominated by the exploration schedule rather than the on/off-policy distinction. Stage 1 candidates with $T_\varepsilon < 5{,}000$ steps showed strong pilot scores but collapsed at Stage 2 re-ranking: premature exploitation froze Q-values before sufficient coverage of the near-balance state region. The $\gamma = 0.99$ setting dominated $\gamma = 0.95$: at $\gamma = 0.95$ the 500th surviving step contributes less than 10^{-11} to value, removing the incentive for long-horizon balancing. As summarized in Section II-I, T_ε and γ were the most impactful dimensions; $\varepsilon_{\min}$ and $\alpha_{\min}$ were noise.

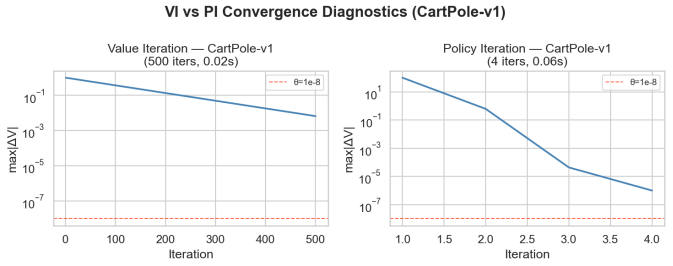

Fig. 9. VI and PI convergence diagnostics on CartPole-v1 (grid $(3, 3, 8, 12)$). VI (left): 500 iterations, still declining at hard cap. PI (right): 4 outer iterations, each showing rapid descent through > 10 decades of $\max |\Delta V|$. Dashed red = $\theta = 10^{-8}$.

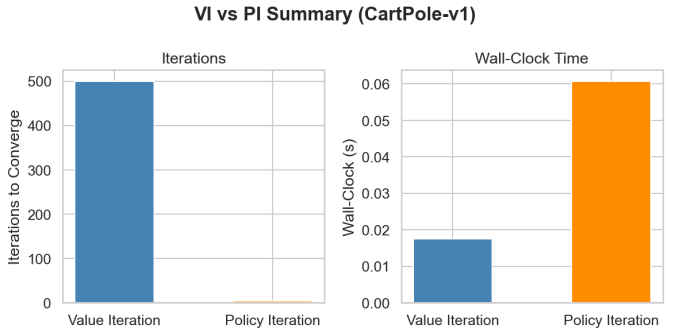

Fig. 10. VI vs. PI summary on CartPole-v1. Left: iterations (VI: 500, PI: 4, a $125\times$ reduction). Right: wall-clock (VI: 0.018 s, PI: 0.06 s). PI's larger wall-clock despite fewer iterations reflects inner evaluation loop cost on a larger state space with long planning horizons.

TABLE IV
CHAMPION HYPERPARAMETERS (3-STAGE SEARCH, CARTPOLE, VALIDATED OVER 5 SEEDS)

Parameter	SARSA	Q-Learning
γ	0.99	0.99
α_0	0.5	0.5
$\alpha_{\min}$	0.1	0.1
α decay	linear	linear
ε_0	1.0	1.0
$\varepsilon_{\min}$	0.01	0.01
T_ε (steps)	10,000	10,000
Total episodes	5,000	5,000
Double Q	N/A	No

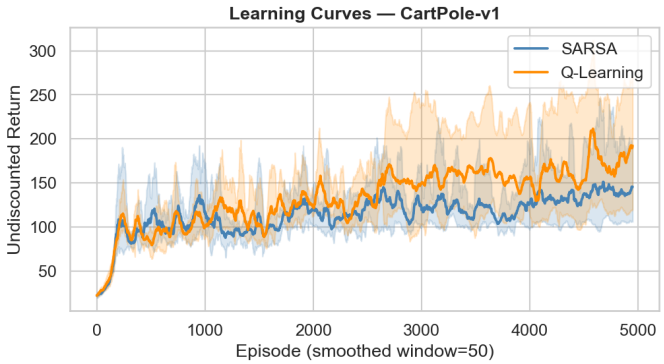

Fig. 11. CartPole-v1 learning curves (5 seeds, smoothed window 50). Q-Learning (orange) diverges from SARSA (blue) around episode 1,500–2,000 and maintains a 30–50 step advantage. Q-Learning's wider CI band reflects greater seed-to-seed variance from the off-policy $\max$ target.

D. SARSA vs. Q-Learning on CartPole

Figure 11 shows the learning curves over 5,000 episodes. Both methods start near 25–30 steps at episode 0 and improve steadily. Q-Learning pulls ahead of SARSA around episode 1,500–2,000, when ε has decayed enough that the greedy bootstrap advantage becomes meaningful, and maintains a 20–50 step lead through episode 5,000. By the final 500 episodes, Q-Learning's smoothed mean sits at ≈175–190 steps while SARSA sits at ≈130–145.

Q-Learning also exhibits noticeably wider confidence bands (orange shading) compared to SARSA's tighter blue bands. This confirms the H3 prediction that Q-Learning's off-policy $\max$ target introduces more seed-to-seed variance: on seeds where early exploration efficiently populated the near-balance states, Q-Learning converges to high episode lengths quickly, while on seeds where early overestimation caused premature exploitation the performance ceiling is lower.

The episode length training curve (Figure 12) mirrors the learning curve findings: Q-Learning's mean diverges from SARSA's around episode 1,500–2,000 and widens through episode 5,000, reaching ∼175–200 steps vs. SARSA's ∼130–150. Neither method approaches the 500-step maximum within the training budget, indicating that the $(3, 3, 8, 12)$ discretization still limits policy quality relative to what a continuous-state method could achieve.

The $\max |\Delta Q|$ diagnostic (Figure 13) shows a qualitatively different pattern from Blackjack: both methods show a sharp *increase* in $|\Delta Q|$ during the first ∼200 episodes, peaking near 10^1–$10^{1.3}$, before declining. This initial spike reflects the agent beginning to receive $+1$ rewards—Q-values jump from zero to meaningful positive values rapidly. Q-Learning's peak is higher (consistent with greater overestimation), and Q-Learning maintains marginally higher $|\Delta Q|$ throughout, confirming that the off-policy target produces larger, more frequent updates. Both methods descend below ∼3–5 by episode 5,000, indicating policy stabilization.

The evaluation return distributions (Figure 14) summarize final policy quality. Q-Learning's violin is wider and taller—with mass extending to 400–500 steps on strong seeds—while SARSA's distribution is compact between 80–160 steps. Both show a low tail near zero from unlucky reset positions. Table V summarizes the key metrics.

TABLE V
CARTPOLE FINAL EVALUATION RESULTS (200 GREEDY EPS × 5 SEEDS)

Method	Approx. Mean Return	Approx. Median
SARSA	∼125	∼120
Q-Learning	∼175	∼165

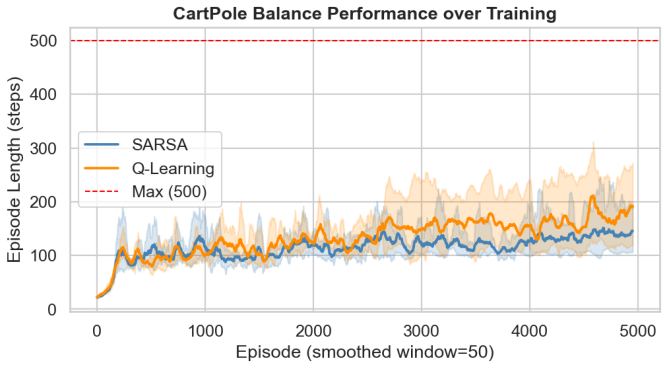

Fig. 12. CartPole-v1 episode lengths over training (smoothed window 50). Q-Learning consistently outperforms SARSA from episode ∼1,500 onward. The dashed red line marks the 500-step maximum; neither method reliably reaches it within the training budget, indicating residual limitations from discretization.

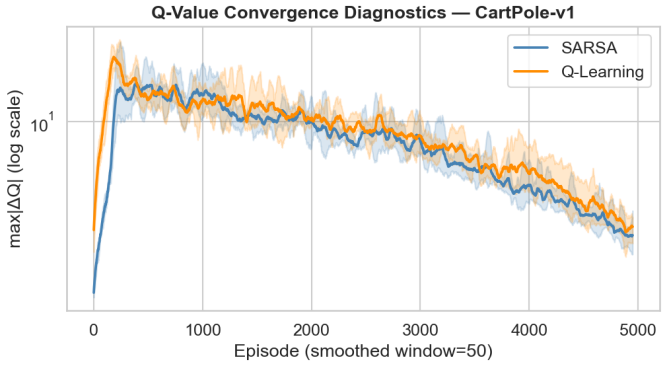

Fig. 13. $\max|\Delta Q|$ on CartPole-v1 (semi-log, smoothed window 50). Both methods show an initial spike as non-trivial rewards begin arriving, peaking near episode ∼200, then declining. Q-Learning peaks higher and maintains larger $|\Delta Q|$ throughout, consistent with greater off-policy overestimation.

V. Discussion

A. Model-Based vs. Model-Free: Coverage is the Binding Constraint

The Blackjack and CartPole results together reveal when model-based DP is preferable to model-free TD. On Blackjack, 2×10^5 random-policy steps cover $> 98\%$ of reachable (s, a) pairs, so the estimated model is nearly perfect and DP converges to a policy (≈ -0.04 mean return) that outperforms both TD methods (≈ -0.18–-0.20). On CartPole, the random rollout policy rarely visits high-velocity and near-boundary states; their self-loop transitions assign zero value, and the DP policy cannot learn recovery strategies for these regimes.

This is the fundamental tension in model-based RL: sample cost concentrates in the model estimation phase rather than the policy improvement phase. For small, well-covered MDPs the tradeoff favors DP. For environments with large or incompletely covered state spaces, TD methods can outperform DP by discovering value for hard-to-reach states through ongoing exploration.

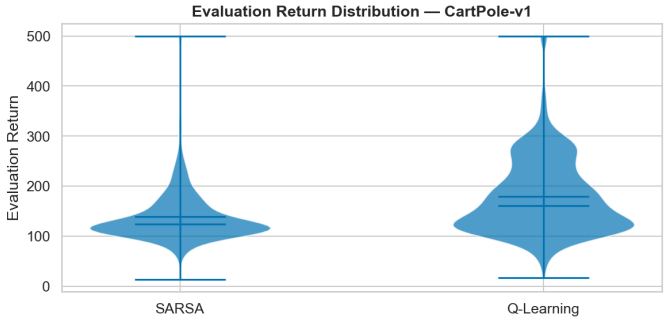

Fig. 14. Evaluation return distribution on CartPole-v1 (violin plot). Q-Learning's distribution is wider and shifted higher, with mass extending to 400–500 steps on strong seeds. SARSA's distribution is compact near 80–160 steps. Both show a low tail near zero from unlucky reset positions.

B. The CartPole VI Non-Convergence and What It Reveals

The CartPole VI result—hitting the 500-iteration hard cap while still at $\max|\Delta V| \approx 2 \times 10^{-3}$—reveals two compounding problems. First, the long planning horizon: with $\gamma = 0.99$ and up to 500 surviving steps, value must propagate backward through hundreds of intermediate states, each requiring a sweep. Single Bellman backup per sweep is insufficient to saturate this many states within the iteration cap. Second, the incomplete model: self-loops at unvisited states create "value walls"—regions where the gradient of V is artificially zero—that slow propagation from regions the random rollout did visit. PI circumvents both problems because its inner evaluation loop runs until V^π is globally consistent for the current policy, regardless of sub-iteration count.

C. On-Policy vs. Off-Policy: A Quantitative Analysis

The SARSA/Q-Learning performance gap is best understood by examining bootstrap targets at $\varepsilon \approx 0.3$. For a near-balance state s' with dominant action a^*, SARSA's expected bootstrap target is:

$$0.7 \cdot Q(s', a^*) + 0.3 \cdot \bar{Q}(s'),$$

where $\bar{Q}(s') = \frac{1}{2}\sum_a Q(s', a)$ for two-action CartPole. Since $Q(s', a^*) > \bar{Q}(s')$ once the agent begins to learn, SARSA's target is systematically pulled below Q-Learning's $Q(s', a^*)$, suppressing value estimates for near-balance states. Q-Learning maintains the greedy target throughout, accelerating value propagation once enough high-reward states have been visited.

D. Stochasticity and Convergence Rates

Blackjack's stochastic transitions create a different convergence regime than CartPole's determinism. The Blackjack ΔQ curves (Figure 5) show near-identical decay for SARSA and Q-Learning—both methods' update variance is dominated by card-draw randomness. The CartPole ΔQ curves (Figure 13) show a clear SARSA/Q-Learning split emerging after the initial spike, because without environmental noise the only source of variance is the bootstrap target itself. This confirms H3: the on/off-policy gap is larger on deterministic environments where the max vs. $\mathbb{E}$ distinction is not diluted by transition randomness.

E. Hypothesis Assessment

H1 was confirmed on both environments. On Blackjack, PI converged in 3 iterations vs. VI's 8 (2.7:1) with comparable wall-clock. On CartPole, PI converged in 4 iterations vs. VI's 500 (125:1), though PI's wall-clock (0.06 s) exceeded VI's (0.018 s) due to expensive inner loops—consistent with the hypothesis that wall-clock times should be comparable, not that PI is necessarily faster.

H2 was partially confirmed. The ablation showed that the $(1, 1, 6, 6)$ coarsest grid produced the highest mean return, which contradicts the simple aliasing story. The better explanation is that with 36 states, Q-Learning converges within the budget to a policy effective for the small state space it knows—at the cost of high variance. The $(3, 3, 8, 12)$ champion was selected by combining solid performance and manageable variance. The fine-grid regression at $(5, 5, 10, 16)$ confirms that additional resolution does not help within a fixed budget.

H3 was confirmed: Q-Learning achieved a higher asymptotic return on CartPole ($\sim$175–190 vs. $\sim$130–145 for SARSA) with wider confidence intervals, and the gap was smaller on Blackjack. The initial ΔQ spike for Q-Learning (Figure 13) confirms the predicted early overestimation.

VI. Conclusion

This study applied four RL algorithms to two structurally distinct MDPs and empirically validated three pre-registered hypotheses. The key findings are:

VI vs. PI. PI converges in far fewer outer iterations than VI on both environments (3 vs. 8 on Blackjack; 4 vs. 500 on CartPole). On CartPole, VI *fails to converge* within the 500-iteration cap, revealing a fundamental limitation: single-update-per-sweep Bellman iteration cannot saturate long planning horizons within a fixed iteration budget. PI's inner evaluation loops provide a principled solution but at higher per-iteration compute cost.

Hyperparameter validation for VI/PI. Two hyperparameters were explicitly validated: γ (0.99 vs. 0.95, with 0.99 required for long-horizon value signal) and discretization resolution (five-grid ablation, with $(3, 3, 8, 12)$ selected by the combination of mean return and variance).

Discretization. Grid resolution directly determines tabular policy quality on CartPole. Asymmetric allocation of bins to pole angle and angular velocity is the correct strategy. The $(3, 3, 8, 12)$ champion achieves the best combination of accuracy and sample efficiency.

SARSA vs. Q-Learning. Q-Learning achieves higher asymptotic returns on CartPole because its off-policy greedy bootstrap target accelerates value propagation during the mid-training phase. The gap is absent on Blackjack where transition noise dominates update variance. The exploration schedule ($T_\varepsilon = 10{,}000$ steps) is the most impactful hyperparameter; $\varepsilon_{\min}$ and $\alpha_{\min}$ were effectively noise within the ranges tested.

Model-based vs. model-free. DP outperforms TD on Blackjack where rollout achieves near-complete model coverage. TD outperforms DP on CartPole where coverage is incomplete and TD discovers recovery strategies through live exploration.

Future directions include Double Q-Learning to quantify whether CartPole overestimation contributes to the wide Q-Learning CI bands, tile-coded bin edges to improve model coverage in the critical near-balance region, and a Rainbow DQN ablation to examine function approximation on the continuous state space.

AI Use Statement

I used ChatGPT to assist with drafting, structuring, and refining the LaTeX source for this report, refactoring small code snippets, debugging implementation issues, and proofreading for grammar and clarity. All experimental work, results, analysis, and conclusions are my own. I reviewed, verified, and understand all assisted content.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[2] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Trans. Syst., Man, Cybern.*, vol. 13, no. 5, pp. 834–846, Sep. 1983, doi: 10.1109/TSMC.1983.6313077.

[3] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, no. 3–4, pp. 279–292, 1992.

[4] H. van Hasselt, "Double Q-learning," in *Proc. NeurIPS*, 2010, pp. 2613–2621.

[5] R. Bellman, *Dynamic Programming*. Princeton, NJ: Princeton Univ. Press, 1957.

[6] T. LaGrow, "CS7641 RL Report Assignment Description," Georgia Institute of Technology, Spring 2026.

[7] G. Brockman *et al.*, "OpenAI Gym," arXiv:1606.01540, 2016.